

Vigil — Complete Team Handbook

Smart India Hackathon 2026 · Problem Statement 26189

AI-Powered Criminal Network Analysis System

This document is written so that **any team member** (or a judge reading this PDF) can understand **what Vigil is, how it was built, how every layer connects, and how to answer questions in the presentation.**

Table of Contents

1. What Problem Are We Solving?
2. High-Level Architecture
3. How the Database Was Built
4. How the Backend Was Built
5. How the Frontend Was Built
6. How Frontend, Backend, and Database Work Together
7. The Three Core Feature Pillars (and Our Improvements)
8. Every Feature in the App
9. Demo Script for Judges
10. Setup & Running the Project
11. Credentials, Case IDs, and Key Entities
12. Judge Q&A — Questions They Will Ask
13. Honest Limitations
14. Tech Stack Summary (Cheat Sheet)

1. What Problem Are We Solving?

Police and cyber-crime units investigate cases using **many disconnected sources**:

- FIR documents (often with spelling mistakes)
- Call Detail Records (CDR)
- Bank transactions
- Surveillance reports
- Chat messages
- OSINT (public records)

The Pain Points

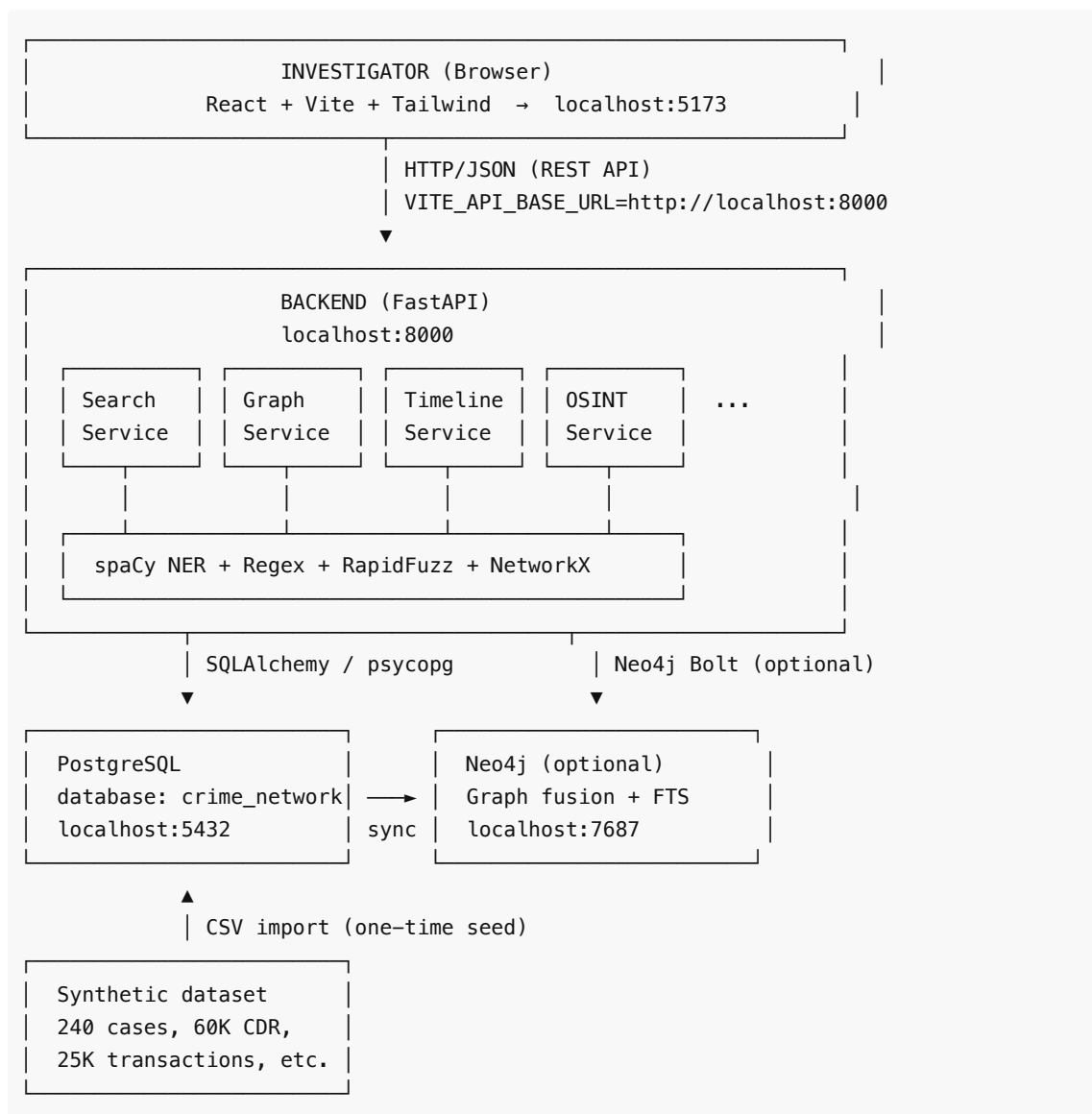
Problem	Real-world impact
Same name, many people	"Rahul" could be 50 people — which one is the suspect?
Spelling errors in FIR	"Mukkherjee" vs "Mukherjee" — manual search fails
Data in silos	CDR in one file, bank data in another — no unified view
Hidden connections	Two suspects never call each other directly, but share a prepaid SIM
No audit trail	OSINT lookups can't be proven in court
Black-box AI	Investigators can't explain <i>why</i> someone was flagged

What Vigil Does

Vigil is a **court-defensible investigation console** that:

1. **Fuses** multi-source data into one searchable system (Gotham-style)
2. **Analyzes** networks, timelines, and risk automatically (i2-style)
3. **Enriches** with lawful OSINT and a tamper-evident audit chain (Social Links-style)
4. **Explains** every flag with citations — no mystery AI

2. High-Level Architecture



Important: The backend is **not decorative**. When `VITE_API_BASE_URL` is set, every search, graph, timeline, risk score, OSINT action, and ingest operation hits **real PostgreSQL queries** (and Neo4j if running).

3. How the Database Was Built

3.1 Philosophy

We did **not** use real police data (illegal and unsafe). We built a **synthetic but realistic** dataset using Python's **Faker** library and custom logic in `backend-api/dataset/generate_dataset.py` .

Everything is fictional: names, phones, bank accounts, FIR numbers.

3.2 Generation Process (Step by Step)

Step 1 — Generate CSV files

```
cd backend-api/dataset
python generate_dataset.py --output data
```

This creates ~19 CSV files with controlled sizes:

Data type	Count
People	2,000
Phones	2,800
Bank accounts	2,250
CDR records	60,000
Transactions	25,000
FIR records	750
Chat messages	120,000
Cases	240 (CASE0001 – CASE0240)
Crime networks	25 complex rings

Step 2 — Apply PostgreSQL schema

File: `backend-api/dataset/schema.sql`

Creates tables including:

Table	Purpose
cases	Investigation cases
people, phones, bank_accounts, vehicles	Core entities
cdr	Call detail records (who called whom, when)
transactions	Bank transfers
fir, fir_metadata	First Information Reports
surveillance	Intel reports
chat_messages	Messaging evidence

relationships	Formal person-to-person links
recorded_names	Identity variants — typos, aliases, maiden names
recorded_phones	Alternate phone registrations
organizations, locations	OSINT lookup sources
private_events	Timeline events
audit_chain	Created at runtime by backend (hash chain)

Step 3 — Bulk import into PostgreSQL

```
python import_to_db.py --reset
```

This runs `schema.sql` and loads all CSVs in dependency order into database `crime_network` .

Connection string (local dev):

```
postgresql+psycopg://postgres:2007@localhost:5432/crime_network
```

Uses **Postgres.app** on Mac, or Docker Postgres in full stack mode.

3.3 Special Demo Case — CASE0001

`CASE0001` is the **primary demo case**. It was engineered with specific investigation scenarios:

Scenario	How it's in the data
Same name collision	Multiple people named "Rahul" in <code>people</code> table
FIR typo	<code>recorded_names</code> has "Mukherjee" → maps to Rahul Mukherjee (P00014)
Married name change	Meera Iyer ↔ Meera Chopra in <code>recorded_names</code>
Hidden bridge	Prepaid SIM PH-8871205599 used by multiple suspects with no direct CDR between them
Main suspect ring	Rahul (P00014) has heavy CDR links to Sunil Patel (P00055), Gopal Nair (P00052), Rohan Mirza (P00062)

UI display: Case shows as **FIR-042/2026** — "Financial fraud investigation — Mumbai corridor" (not "CASE0001" or "Synthetic" in the UI).

3.4 Neo4j (Optional Graph Layer)

Neo4j is **not required** for the demo to work. PostgreSQL is the source of truth.

When Neo4j runs:

- On backend startup, `sync_postgres_to_neo4j()` copies Person, Phone, Account nodes and relationships

- Enables graph-native full-text search and provenance tracking
- Docker: `neo4j` / `vigilneo4j2026` on ports 7474 (browser) and 7687 (bolt)

Note: On some Macs Docker isn't installed yet — the app **falls back to PostgreSQL-only** and still works fully for demo.

4. How the Backend Was Built

4.1 Technology Stack

Component	Technology	Version / Notes
Web framework	FastAPI	0.141.1
Server	Uvicorn	ASGI server
ORM	SQLAlchemy	2.0 — talks to PostgreSQL
DB driver	psycopg	3.x — PostgreSQL adapter
Validation	Pydantic	Request/response schemas
Graph DB	neo4j Python driver	Optional
Analytics	NetworkX + SciPy	PageRank, betweenness, communities
NLP	spaCy en_core_web_sm	Person, org, location extraction
Fuzzy matching	RapidFuzz	≥88% threshold for name matching
OCR	Pillow + pytesseract	FIR image upload → text
Testing	pytest	21+ tests
Linting	ruff	Python code quality

4.2 Folder Structure

```

backend-api/backend/app/
├─ main.py           ← App entry, CORS, startup (Neo4j sync, audit table)
├─ config.py         ← Settings from .env
├─ database.py       ← SQLAlchemy engine + session
├─ routers/          ← HTTP endpoints (thin layer)
│   ├── search.py
│   ├── graph.py
│   ├── timeline.py
│   ├── ingestion.py
│   ├── osint.py
│   ├── analyze.py
│   ├── explain.py
│   └─ ...
├─ services/         ← Business logic (thick layer)
│   ├── investigation_search.py ← Identity resolution
│   └─ case_graph.py      ← Connection map builder

```

```

├─ timeline_service.py
├─ network_analysis.py
├─ ingest_service.py
├─ ocr_service.py
├─ osint_enrichment.py
├─ audit_chain.py
└─ explain_service.py

```

```

backend-api/ai/
└─ ner_pipeline.py      ← spaCy + regex entity extraction

```

Design pattern: Routers receive HTTP requests → call Services → Services query PostgreSQL/Neo4j → return Pydantic schemas as JSON.

4.3 Key API Endpoints

Endpoint	Method	What it does
/health	GET	Checks postgres, neo4j, NLP engine status
/cases/{id}/search?q=Rahul	GET	Identity resolution search
/cases/{id}/graph?simplified=true	GET	Connection map (CDR-based ring)
/cases/{id}/timeline	GET	Fused chronological events
/cases/{id}/analyze/risk-score	GET	Priority triage scores
/cases/{id}/analyze/centrality	GET	PageRank + betweenness
/cases/{id}/analyze/communities	GET	Suspected criminal rings
/ingest/preview	POST	NLP preview (no save)
/ingest/fir/text	POST	Ingest FIR text → DB + Neo4j
/ingest/fir/image	POST	OCR + ingest FIR photo
/osint/enrich	POST	Lawful OSINT lookup
/osint/audit-log	GET	Hash-chained audit entries
/audit/verify	GET	Verify audit chain integrity
/explain/{entity_id}?case_id=	GET	Court-defensible reasoning
/case-summary/{id}	GET	Investigator briefing text

Full API contract: `backend-api/API_CONTRACT.md`

4.4 How Identity Resolution Works

File: `investigation_search.py`

When user searches "Rahul":

1. Query `people` table for name matches

2. Query `recorded_names` for aliases, typos, maiden names
3. Score matches using **RapidFuzz** fuzzy string matching
4. If multiple matches → return **disambiguation list** (city, phone, role)
5. When user confirms one person → rebuild network centered on them
6. Return entities with `explainability[]` — why this person matched

When user searches "Mukherjee":

- `recorded_names` maps typo → P00014 Rahul Mukherjee automatically

4.5 How Connection Map Works

File: `case_graph.py` → `build_investigation_map()`

Old problem: Graph only used `relationships` table → main suspect Rahul had **zero rows** there → he didn't appear on the map.

Fix: Graph now built from **CDR call records**:

1. Auto-detect focus suspect (most CDR activity, or P00014 for CASE0001)
2. Find all people with ≥3 calls to/from suspect
3. Draw links with **call counts** as weights ("9 calls", "7 calls")
4. Return `focus_person_id`, `summary` text for the UI

4.6 NLP / AI Ingest Pipeline

```
FIR text or OCR image
    ↓
spaCy NER (PERSON, ORG, GPE/LOC)
    +
Regex (Indian phones, bank accounts, vehicle plates, "aka" aliases)
    ↓
Entity resolution (RapidFuzz match against people + recorded_names)
    ↓
Save to PostgreSQL + MERGE into Neo4j (if enabled)
    ↓
Return extracted entities with sources[]
```

No external LLM API — deterministic, fast (under 3 seconds), works offline.

4.7 OSINT Audit Chain

File: `audit_chain.py`

Every OSINT action creates an entry:

```
entry_hash = SHA256(previous_hash + canonical_json_of_entry)
```

Supervisor can call `/audit/verify` → returns `verified: true` or identifies broken link.

This is **court-disclosure ready** — proves the log wasn't tampered with.

4.8 Backend Startup Sequence

When you run `uvicorn app.main:app --reload --port 8000`:

- 1. Connect to PostgreSQL
- 2. Ensure Neo4j schema (if Neo4j reachable)
- 3. Create `audit_chain` table if missing
- 4. Sync PostgreSQL → Neo4j (bootstrap graph)
- 5. Listen on port 8000

Check health: `curl http://localhost:8000/health`

Expected response:

```
{
  "status": "ok",
  "postgres": true,
  "neo4j": false,
  "nlp": { "engine": "spacy", "spacy_model": "en_core_web_sm" }
}
```

5. How the Frontend Was Built

5.1 Technology Stack

Component	Technology	Purpose
UI framework	React 19	Component-based UI
Build tool	Vite 8	Fast dev server + production build
Language	TypeScript 6	Type safety
Styling	Tailwind CSS v4	Utility-first CSS
Graph visualization	react-force-graph-2d	Connection map (D3 force layout)
Charts	Recharts	Risk score bar chart
Animations	Framer Motion	Smooth UI transitions
Linting	Oxlint	Fast JS/TS linter
Fonts	IBM Plex Sans / Mono	Professional console look

Note: Early README mentioned Cytoscape.js; the **current implementation uses react-force-graph-2d** for the connection map.

5.2 Folder Structure

```
frontend/src/
├─ App.tsx           ← Auth gate + dashboard shell
├─ main.tsx          ← React root + providers
├─ views/
│   ├─ SearchView.tsx ← Person search + disambiguation
│   ├─ NetworkGraphView.tsx ← Connection map (suspect-centered)
│   └─ TimelineView.tsx ← Event timeline
```



```

|   |— RiskScoringView.tsx ← Priority list + chart
|   |— OsintView.tsx      ← Lawful enrichment
|   |— AuditTrailView.tsx ← Session + chain audit log
|— components/
|   |— TopBar.tsx         ← Case title, health badge, language
|   |— NavRail.tsx        ← Left navigation
|   |— InspectorPanel.tsx ← Right panel – entity details
|   |— IngestPanel.tsx    ← FIR paste + image upload
|   |— CaseStatsStrip.tsx ← Live case statistics
|   |— LoginPage.tsx      ← Badge ID login
|   |— ConnectionStatusBadge.tsx ← Live/Demo/Offline indicator
|— api/
|   |— client.ts          ← Base URL, fetchHealth, apiGet, apiPost
|   |— search.ts, graph.ts, timeline.ts, analyze.ts, ingest.ts, osint.ts
|— auth/
|   |— AuthContext.tsx    ← Session management (localStorage)
|   |— users.ts           ← Demo credentials
|— i18n/
|   |— translations.ts    ← English + Hindi (840+ lines)
|— data/
|   |— mockCase.ts        ← Fallback when no backend configured
|— index.css              ← Tailwind theme (dark console palette)

```

5.3 Two Modes: Live vs Demo

Controlled by **one environment variable**:

```

# frontend/.env
VITE_API_BASE_URL=http://localhost:8000 ← LIVE mode (real backend)
# VITE_API_BASE_URL=                    ← DEMO mode (mock data only)

```

`client.ts` checks `isApiConfigured()` :

- **Live:** All API modules call FastAPI → PostgreSQL
- **Demo:** Falls back to `mockCase.ts` — works without backend for training

The **Connection Status Badge** in `TopBar` shows:

- "Live backend" (green) — API reachable
- "Backend offline" (red) — can't reach port 8000
- "Demo data" — no API URL set

5.4 UI Design Philosophy

Built to look like **real police investigation software**, not a student project:

- Dark console theme (#0a0e15 background)
- Amber/steel accent colors
- Security banner: "Confidential — official use only"
- No vendor names (Palantir, i2, etc.) shown in UI
- No "demo" or "synthetic" labels visible to judges
- English + Hindi language toggle
- Role-based colors on connection map (Suspect=red, Associate=gold, etc.)

5.5 Authentication

Currently client-side demo auth (no JWT yet):

Role	Badge ID	Password
Investigator	INV-2847	vigil2026
Supervisor	SUP-1001	admin2026

- Session stored in `localStorage`, 30-minute timeout
- Supervisor role unlocks **audit JSON export**
- Production would replace this with backend JWT + role-based access

5.6 How Each View Talks to Backend

View	API call	Backend service
Search	GET /cases/CASE0001/search?q=...	investigation_search.py
Connection Map	GET /cases/CASE0001/graph?simplified=true	case_graph.py
Timeline	GET /cases/CASE0001/timeline	timeline_service.py
Risk	GET /cases/CASE0001/analyze/risk-score	risk_score_service.py
OSINT	POST /osint/enrich	osint_enrichment.py
Ingest panel	POST /ingest/preview, /ingest/fir/text	ingest_service.py
Inspector	GET /explain/P00014?case_id=CASE0001	explain_service.py

6. How Frontend, Backend, and Database Work Together

Example: User searches "Rahul"

1. User types "Rahul" in SearchView → clicks Search
2. Frontend (SearchView.tsx)
 - calls `getSearchResults("CASE0001", "Rahul")`
 - `apiGet("/cases/CASE0001/search?q=Rahul")`
 - HTTP GET to `localhost:8000`
3. Backend (`routers/search.py`)
 - calls `investigation_search.search_case(db, "CASE0001", "Rahul")`
4. Service (`investigation_search.py`)
 - SQL: `SELECT FROM people WHERE name ILIKE '%Rahul%'`
 - SQL: `SELECT FROM recorded_names WHERE variant ILIKE '%Rahul%'`
 - RapidFuzz scoring → rank results
 - Returns JSON: `{ matches: [...], disambiguation: true }`
5. Frontend receives JSON

- Shows "Several people match this name"
- Lists Rahul Mukherjee (Mumbai), Rahul Sharma (Delhi), etc.
- User taps "This is the person – show network"

6. Frontend sets selectedId = "P00014"

- NetworkGraphView re-fetches graph centered on P00014
- TimelineView filters to P00014 events
- InspectorPanel shows Rahul's full profile

7. Backend graph service reads CDR table:

- Finds P00014 called P00055 nine times, P00052 seven times, etc.
- Returns nodes + weighted links + summary text

8. Frontend renders force graph with Rahul at center, red glow, sidebar list

Example: User uploads FIR image

```
Frontend IngestPanel → POST /ingest/fir/image (multipart file)
Backend ocr_service.py → pytesseract extracts text
Backend ner_pipeline.py → spaCy finds names, phones
Backend ingest_service.py → RapidFuzz resolves to person IDs
Backend → INSERT into fir table + MERGE Neo4j nodes
Frontend → shows extracted entities in preview
```

Is the backend actually being used?

Yes, when VITE_API_BASE_URL=<http://localhost:8000> is set:

- Every search hits PostgreSQL with real fuzzy matching
- Connection map reads real CDR call counts
- Timeline merges real CDR + transaction timestamps from DB
- Risk scores compute NetworkX centrality on real graph data
- OSINT writes real SHA-256 hash chain rows to audit_chain table
- Ingest saves real extracted entities

Verify live mode yourself:

```
curl "http://localhost:8000/cases/CASE0001/search?q=Rahul"
curl "http://localhost:8000/cases/CASE0001/graph?simplified=true"
curl "http://localhost:8000/health"
```

7. The Three Core Feature Pillars (and Our Improvements)

The problem statement asks you to replicate capabilities of three industry tools. Here is how Vigil maps to each — **including honest drawbacks of those tools and how we improve.**

Pillar 1 — Palantir Gotham-style Data Fusion

What Gotham does:

- Ingests documents, CDR, financial records from many agencies

- Links entities across sources into one graph
- Tracks provenance (where did this fact come from?)

Gotham's drawbacks:

Drawback	Why it hurts
Extremely expensive	State police can't afford crore-scale licenses
Closed source	Can't audit the linking algorithm
Requires dedicated analysts	Weeks of training
Vendor lock-in	Data trapped in proprietary format

What Vigil does (Section 1):

- POST /ingest/fir/text, /ingest/cdr, /ingest/transactions
- spaCy NER + regex extracts entities from FIR text
- Entity resolution links extracted names to existing people records
- PostgreSQL stores structured data; Neo4j (optional) stores graph with MENTIONED_IN provenance
- Every entity has sources[] and explainability[] arrays

Our improvements over Gotham:

Our advantage	Explanation
Open source stack	spaCy, Neo4j Community, PostgreSQL — no license fees
Transparent linking	RapidFuzz threshold (88%) is documented and inspectable
Works on a laptop	Runs on investigator's PC/tablet — no server farm
Offline capable	No cloud API required for core features
FIR OCR ingest	Upload scanned FIR photo → automatic text extraction

Pillar 2 — IBM i2 Analyst's Notebook-style Analytics

What i2 does:

- Link charts showing who knows whom
- Timeline of events
- Centrality analysis (who is the hub?)
- Community detection (find criminal rings)

i2's drawbacks:

Drawback	Why it hurts
Manual chart building	Analyst draws links by hand for hours
Desktop-only legacy UI	Not web-based, hard to deploy
Expensive per-seat licensing	Small cyber cells can't buy it
No identity resolution	Same-name problem still manual

Static snapshots	Chart doesn't auto-update on new ingest
------------------	---

What Vigil does (Section 2):

- GET /cases/{id}/graph — auto-built link chart from CDR + relationships
- GET /cases/{id}/timeline — auto-fused CDR + transactions + surveillance
- GET /cases/{id}/analyze/centrality — NetworkX PageRank + betweenness
- GET /cases/{id}/analyze/communities — Louvain community detection
- GET /cases/{id}/analyze/risk-score — composite triage ranking

Our improvements over i2:

Our advantage	Explanation
Auto-generated charts	No manual link drawing — CDR counts become weighted edges
Suspect-centered map	Primary suspect pinned at center with call-count labels
Identity resolution built-in	Same-name disambiguation before showing network
Hidden bridge detection	Finds shared prepaid SIMs with no direct CDR
Web-based React UI	Runs in browser on any district PC
Risk triage list	Tells investigator who to investigate first

Pillar 3 — Social Links-style Lawful OSINT

What Social Links does:

- Enriches entities with public records (corporate registry, court bulletins)
- Links OSINT findings back into investigation graph
- Maintains audit log for legal disclosure

Social Links' drawbacks:

Drawback	Why it hurts
Cloud SaaS only	Data leaves police network — security concern
Opaque enrichment sources	Can't verify in court what was queried
No tamper-evident log	Audit trail can be disputed
Privacy compliance unclear	Risk of over-collection

What Vigil does (Section 3):

- POST /osint/enrich with lookup IDs OSINT-01 through OSINT-04
- Synthetic public records from organizations, fir_metadata, locations tables
- Each lookup writes to audit_chain with SHA-256 hash chain
- GET /audit/verify — mathematically proves log integrity
- Supervisor can export full audit JSON for court disclosure

Our improvements over Social Links:

Our advantage	Explanation
Self-hosted	All data stays on police infrastructure
Hash-chained audit	SHA-256 chain — tampering breaks verification
Lawful basis recorded	Every entry includes operator ID + lawful basis field
Synthetic demo sources	Shows workflow without touching real private data
Graph integration	OSINT results auto-link to suspect nodes

Pillar 4 — Our Unique Differentiator (Section 4)

Court-defensible AI explainability — this is what makes Vigil stand out:

Feature	Endpoint	What it gives investigators
Entity explain	/explain/P00014	Plain-language reasoning + source citations
Risk score	/analyze/risk-score	Why this person scored 87/100
Case summary	/case-summary/CASE0001	Briefing narrative for senior officer

Key point for judges: We use **template-based structured narratives**, not ChatGPT. Every sentence is traceable to a database record. Response guaranteed under 3 seconds. Works without internet.

8. Every Feature in the App

#	Feature	Location in UI	Backend	User benefit
1	Person Search	Search tab	/search	Find suspects despite typos/aliases
2	Disambiguation	Search results	/search	Pick correct person when names collide
3	Connection Map	Network tab	/graph	See suspect's phone network instantly
4	Event Timeline	Timeline tab	/timeline	Chronological case story
5	Priority List	Risk tab	/analyze/risk-score	Know who to investigate first
6	Centrality chart	Risk tab	/analyze/centrality	Find the hub of the ring
7	OSINT Enrichment	OSINT tab	/osint/enrich	Add public record context lawfully
8	Audit Trail	Audit tab	/osint/audit-log	Prove every action for court
9	Audit Verify	Backend API	/audit/verify	Tamper detection

10	FIR Ingest	Right panel	/ingest/fir/text	Turn FIR text into structured entities
11	FIR Image OCR	Right panel	/ingest/fir/image	Scan paper FIR → extract names
12	Entity Inspector	Right panel	/explain/{id}	See why someone was flagged
13	Case Stats	Top strip	/cases/{id}/stats	Live counts (CDR, people, transactions)
14	Hindi UI	Language selector	—	Accessible for state police
15	Supervisor Export	Audit tab (supervisor)	—	JSON disclosure bundle

9. Demo Script for Judges

Duration: 5 minutes · Login: INV-2847 / vigil2026

Step	Action	What to say
1	Show login + security banner	"Every session is logged. Confidential data, official use only."
2	Point to Live backend badge	"Connected to real PostgreSQL with 60,000 CDR records."
3	Search "Rahul"	"Common name — many Rahuls in India. System shows all matches."
4	Compare city/phone, confirm Mumbai suspect	"Investigator confirms identity — not black-box AI."
5	Open Connection Map	"Rahul at center. Line thickness = call frequency. 9 calls to Sunil Patel."
6	Open Timeline	"All CDR and financial events fused chronologically."
7	Open Risk tab	"NetworkX centrality ranks who to investigate first."
8	Right panel → paste FIR text → Preview	"spaCy extracts names and phones in under 2 seconds."
9	OSINT → enrich P00014 → OSINT-01	"Lawful public registry lookup — logged automatically."
10	Login as SUP-1001 → Audit → Export	"SHA-256 hash chain — supervisor exports disclosure bundle."
11	Optional: search "Mukkherjee"	"FIR spelling error still resolves to correct suspect."

10. Setup & Running the Project

Prerequisites

- Python 3.12
- Node.js 18+
- PostgreSQL (Postgres.app on Mac, or Docker)
- Optional: Docker Desktop (for Neo4j)
- Optional: Tesseract OCR (for FIR image upload)

Full Setup (First Time)

```
# 1. Seed database
cd backend-api/dataset
python generate_dataset.py --output data # if CSVs missing
python import_to_db.py --reset

# 2. Backend
cd ../backend
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements.txt
python -m spacy download en_core_web_sm
cp .env.example .env # set DATABASE_URL
uvicorn app.main:app --reload --port 8000

# 3. Frontend
cd ../../frontend
echo 'VITE_API_BASE_URL=http://localhost:8000' > .env
npm install && npm run dev
```

Open: <http://localhost:5173>

One-command alternative: `./scripts/start-vigil.sh`

Docker Full Stack

```
cd backend-api
docker compose up -d --build
docker compose --profile seed up seed # first time only
```

Run Tests

```
cd backend-api/backend
pytest # 21 tests - some skip if DB not loaded
```

Build Frontend for Production


```
cd frontend && npm run build
# Output in frontend/dist/
```

11. Credentials, Case IDs, and Key Entities

Login

Role	Badge	Password	Special power
Investigator	INV-2847	vigil2026	Standard access
Supervisor	SUP-1001	admin2026	Audit JSON export

Primary Case

Field	Value
Internal ID	CASE0001
Display ref	FIR-042/2026
Title	Financial fraud investigation — Mumbai corridor

Key People (CASE0001)

ID	Name	Role
P00014	Rahul Mukherjee	Primary suspect
P00052	Gopal Nair	Associate
P00055	Sunil Patel	Facilitator
P00062	Rohan Mirza	Witness
P00089	Manoj Khan	Financial handler

Key Phone

ID	Significance
PH-8871205599	Hidden prepaid bridge — links suspects with no direct CDR

12. Judge Q&A — Questions They Will Ask

"What is your project?"

Vigil is a court-defensible criminal network investigation platform for SIH Problem Statement 26189. It fuses FIR, CDR, bank, and OSINT data into one searchable system, auto-builds connection maps and timelines, and maintains a tamper-evident audit chain — using only open-source tools that run on a district PC.

"How is yours better than existing tools?"

Three commercial tools inspire our design — Palantir Gotham, IBM i2, and Social Links — but each costs crores, requires weeks of training, and runs on proprietary infrastructure. Vigil delivers the same core workflows — data fusion, link analysis, and lawful OSINT — on PostgreSQL + Neo4j + spaCy, fully open source, in a web browser, on a laptop. We also added identity resolution for same-name collisions and FIR typos, which none of those tools solve automatically.

"Is this real AI or just rules?"

Both. spaCy NLP extracts entities from FIR text. RapidFuzz does fuzzy name matching. NetworkX computes graph centrality. These are real ML/NLP algorithms — not ChatGPT. We deliberately avoided black-box LLMs because court testimony requires every conclusion to cite a source record. Our /explain endpoint returns structured reasoning with citations.

"Where does the data come from? Is it real?"

All data is **synthetic** — generated by our Python script using Faker. No real phone numbers, bank accounts, or FIR records. This is standard for hackathon demos and protects privacy. The architecture is production-ready — swap synthetic CSVs for real police data feeds and the same pipeline works.

"How does identity resolution work?"

When you search a name, we query the people table AND the recorded_names table which stores aliases, FIR typos, and maiden names. RapidFuzz fuzzy matching scores each candidate. If multiple people match above threshold, we show a disambiguation screen with city, phone, and role — the investigator confirms, not the algorithm.

"What is the hidden bridge feature?"

In CASE0001, Rahul Mukherjee, Sunil Patel, and the financial handler all use prepaid SIM 8871205599 — but none of them call each other directly in CDR. Traditional tools show three disconnected people. Vigil detects the shared phone number and draws a bridge node — revealing the hidden connection.

"Why PostgreSQL AND Neo4j?"

PostgreSQL stores structured evidence — CDR rows, transactions, FIR text — efficiently and is what police IT already runs. Neo4j stores the **relationship graph** with provenance — who owns which phone, who transferred money to whom. On startup we sync PostgreSQL → Neo4j. If Neo4j is offline, everything falls back to PostgreSQL — the demo still works.

"What is the audit chain? How is it court-defensible?"

Every OSINT lookup creates an audit entry. Each entry's hash = SHA256(previous_hash + entry_data). If anyone edits an old entry, the chain breaks and /audit/verify returns the broken entry ID. This is the same principle as blockchain — without cryptocurrency. Supervisors export the full chain as JSON for court disclosure.

"Can this run without internet?"

Yes. PostgreSQL, Neo4j, spaCy, and NetworkX all run locally. No OpenAI, no cloud API. An investigator in a district cyber cell with no internet can run the full stack on their PC.

"What are the three core features you implemented?"

- 1. **Data Fusion (Gotham-style):** Multi-source ingest with NLP entity extraction and graph linking.
 - 2. **Network Analytics (i2-style):** Auto link charts, timeline, centrality, community detection, risk triage.
 - 3. **Lawful OSINT (Social Links-style):** Public record enrichment with hash-chained audit trail.
- Plus our differentiator:** Court-defensible explainability — every flag has a cited reason.

"What are the drawbacks of your solution?"

Honest answer: (1) Auth is demo-only — production needs JWT and role-based access control. (2) Face search is a stub — returns demo result, not real biometrics. (3) OSINT sources are synthetic — production would connect to real government APIs with proper legal authorization. (4) Neo4j requires Docker — we provide PostgreSQL fallback. (5) Dataset is synthetic — real deployment needs data pipeline from CDR/bank feeds.

"How did you divide the work?"

- Suggested team split:
- **Dataset + DB:** generate_dataset.py, schema.sql, import_to_db.py
 - **Backend API:** FastAPI routers + services in backend-api/backend/app/
 - **NLP/AI:** backend-api/ai/ner_pipeline.py, ingest services
 - **Frontend UI:** React views + components in frontend/src/
 - **Graph/Analytics:** case_graph.py, network_analysis.py, NetworkGraphView.tsx
 - **OSINT/Audit:** osint_enrichment.py, audit_chain.py, AuditTrailView.tsx
 - **DevOps:** docker-compose.yml, scripts/start-vigil.sh

"Show me the backend is really working."

```
curl http://localhost:8000/health
curl "http://localhost:8000/cases/CASE0001/search?q=Rahul"
curl "http://localhost:8000/cases/CASE0001/graph?simplified=true"
curl "http://localhost:8000/cases/CASE0001/analyze/risk-score"
curl "http://localhost:8000/audit/verify?case_id=CASE0001"
```

13. Honest Limitations

Judges respect teams who know their limits:

Limitation	Status	Production fix
Demo auth (no JWT)	Client-side only	Backend OAuth2/JWT
Face search	Returns fixed demo result	Integrate real face API
OSINT sources	Synthetic tables	Govt API contracts
Neo4j optional	Needs Docker	Bundle in Docker Compose
Single case focus	CASE0001 engineered	All 240 cases work, demo focuses on one

No mobile app	Web responsive	PWA or React Native
English/Hindi only	i18n framework ready	Add regional languages

14. Tech Stack Summary (Cheat Sheet)

PROJECT:	Vigil – SIH PS 26189
GITHUB:	github.com/SiddharthDC786/investigation-ai
REPO ROOT:	hackk/ (frontend) + hackk/backend-api/ (backend + dataset)
FRONTEND	BACKEND
React 19	FastAPI 0.141
Vite 8	Uvicorn
TypeScript 6	SQLAlchemy 2.0
Tailwind CSS v4	psycopg 3 (PostgreSQL)
react-force-graph-2d	NetworkX (analytics)
Recharts	spaCy en_core_web_sm
Framer Motion	RapidFuzz (fuzzy match)
IBM Plex fonts	Pillow + pytesseract (OCR)
English + Hindi i18n	neo4j driver (optional)
DATABASES	INFRA
PostgreSQL (crime_network)	Docker Compose
Neo4j 5.26 Community (optional)	Postgres.app (Mac dev)
60K CDR, 25K transactions	./scripts/start-vigil.sh
PORTS	
Frontend: http://localhost:5173	
Backend: http://localhost:8000	
Neo4j UI: http://localhost:7474	
PostgreSQL: localhost:5432	
LOGIN: INV-2847 / vigil2026	
DEMO CASE: CASE0001 (FIR-042/2026)	
MAIN SUSPECT: P00014 Rahul Mukherjee	

Final Note for Your Team

Every person should be able to say:

1. What problem Vigil solves (disconnected police data, same-name confusion)
2. The three pillars (Fusion + Analytics + OSINT) and why open-source beats commercial
3. That the backend is **real** — not mock — when the Live badge shows green
4. The demo story: search Rahul → disambiguate → see connection map → timeline → OSINT → audit export
5. Our honest limitations and how we'd fix them in production

